A42 supplement: SA74 runs (October 31, 2026)

These files reproduce the additions to A42 in Sections 3, 4.2, 4.4, 4.6, 5, 5.1 and 7. Python 3.10 or later with NumPy and SciPy. The scripts import the first passes' engine, A42_Rule_Search.py, saved beside them as a42_rule_search.py, and use its interval choice and its three tests unchanged. Paths to the run folder are set at the top of A42_SA74_Classify.py and A42_SA74_Numbers.py.

The protocol

A42_SA74_Protocol.txt fixed the four-dimension criterion and the runs before they ran. Its SHA-256 is 04eca2085b16830e1386ed165bab6961e1043d76a6681051369c3264c9865ea5, recorded at 2026-10-31T07:20:25Z (A42_SA74_Protocol.sha256). Runs outside it are marked exploratory: transitive percolation tuned to read four on the ordering fraction, originary percolation, the de Sitter comparison, the rule listed in another order, and the transfer test of the step program.

Two departures from the protocol's wording, both reported here: item 6(a) named the ten long-run rules and "four other near misses"; the runs cover all 27 rules of the first pass's long runs, a superset. Item 6(c) named a recheck with up to three further updates; it covers the 27 long-run rules and the first 1,691 of the 2,258 rules that failed with two.

Files

A42_SA74_Extensions.py: noise bands (`bands`), sprinkled link counts (`links2d`), update orders (`orders`), transitive and originary percolation (`tp`, `op`, `tune`), de Sitter tables (`ds`), the critical-pair check (`ci`) and a stricter version that tracks relation identity (`cistrict`).

A42_SA74_Classify.py: applies the protocol: bands interpolated in log N, z-scores, the candidate rule.

A42_SA74_Numbers.py: collects every number the paper quotes into sa74_numbers.json.

Results: bands.json, links_sprinkled.json, orders_oldest/newest/random1/random2.jsonl, orders_pairB.jsonl, ci_22_32.jsonl (two further updates, all 2,935 rules), ci_steps3.jsonl (three, partial), ci_long27_steps3.json, ci_long_sample.json, tp.jsonl (the protocol's grid), tune.jsonl and tp_tuned.jsonl, op.jsonl, ds_table.json, ds_big.json, sa74_numbers.json.

A42_Step_Rule/A42_Step_Rule_Transfer.py and its results: the frozen 451-byte A42_Step_Rule.py, unchanged (SHA-256 c964c768...), run on an expanding 1+1 universe against an independent continuum integration.

How to reproduce

python3 A42_SA74_Extensions.py bands bands.json python3 A42_SA74_Extensions.py links2d links_sprinkled.json python3 A42_SA74_Extensions.py orders long27_rules.json orders_oldest.jsonl oldest (also newest, random1, random2) python3 a42_rule_search.py enumerate rules_22_32.json python3 A42_SA74_Extensions.py ci rules_22_32.json ci_22_32.jsonl --steps 2 python3 A42_SA74_Extensions.py tp tp.jsonl N p seed (the protocol's grid: N 1000 to 8000, p 0.0003 to 0.3, seeds 1 and 2) python3 A42_SA74_Extensions.py tune tune.jsonl N 4.0 tp (then tp at the tuned p, seeds 1 and 2; N 2000 to 64000) python3 A42_SA74_Extensions.py op op.jsonl N p 1 python3 A42_SA74_Extensions.py ds ds_table.json python3 A42_SA74_Numbers.py python3 A42_Step_Rule/A42_Step_Rule_Transfer.py

Runs are deterministic: sprinklings and percolation use seeded NumPy generators, the random update orders seeded Python generators.

The compiled engine (SA74 T2)

A42_SA74_tp_big.c follows transitive percolation held at four on the ordering fraction to two million elements (`gcc -O2 -o tp_big A42_SA74_tp_big.c -lm`; driver A42_SA74_tp_big_drive.py; results tp_big_trajectory.jsonl). It reads the largest interval: the longest chain exactly, the ordering fraction from 3,000 sampled ancestor counts, and the midpoint from the 600 elements that bottom-k sketches rank highest, each counted exactly (environment SKETCH=1 MIDALL=1). At 16,000 elements it matches the exact readings of A42_Rule_Search.py on the same causal sets (dump with DUMP=file), and at 128,000 it finds the same midpoint as checking every element. `tp_big sprinkle 4 N seed` reads sprinkled four-dimensional intervals with all three tests exactly (tp_big_sprinkle4.jsonl).

The known laws on a supplied lattice (SA74 T5)

A42_SA74_Supplied_Lattice.py backs Section 8.7 and Table 12 (`python3 A42_SA74_Supplied_Lattice.py A38_Symbolic_Benchmark_Spec.txt A42_SA74_Supplied_Lattice_Results.json`, NumPy only, about two minutes). It does four things. It compresses the A38 statement of the Standard Model with general relativity with the A38 settings (raw LZMA2, preset 9 extreme), alone and with three blocks printed in the script and in the results: the lattice form of each field, a completion by declared data (a free scalar for dark matter and five numbers for the initial state) and a completion by the SMASH mechanisms, whose inputs are declared for the count at values inside the ranges its authors give. It estimates the digits a physical lattice of Planck spacing needs in the bare

cosmological constant and Higgs mass parameter, from the statement's own inputs. It builds the naive, Wilson and overlap Dirac operators on two-dimensional U(1) lattices of 16 by 16 and 24 by 24 sites, in twelve backgrounds of topological charge -2 to 3, smooth and with seeded noise, and reports doublers, the Ginsparg-Wilson relation and the overlap index against the measured charge. It checks the anomalies of the statement's fermions per family, with 16 and with 15 Weyl fermions, and the photon's dispersion on a four-dimensional hypercubic lattice against the LHAASO limit on quadratic dispersion.

Fermions as forms (SA74 T7)

A42_SA74_Forms.py backs the paragraph on matter in Section 8.7 (`python3 A42_SA74_Forms.py A42_SA74_Forms_Results.json`, NumPy and SciPy, about five minutes). It builds Kähler-Dirac fermions, whose operator d - d† acts on cochains and needs no local frames, on two kinds of triangulation. The first is two-dimensional causal triangulations of a torus: slice lengths are sampled by Metropolis with the strip counts of two-dimensional causal triangulations (Ambjørn and Loll, 1998), strips are drawn uniformly given the lengths, and the triangles are equilateral with circumcentric Hodge stars; a regular triangular torus is the control. It reports the exact zero modes against the Betti numbers and the count of low modes against Weyl's law. The second is the Freudenthal triangulation of a periodic 3^4 lattice with random positive Hodge weights, standing in for a bent geometry, where it reports the zero modes of each Hodge Laplacian against the Betti numbers of the 4-torus.

Quantum Regge gravity on the supplied lattice (SA74 T10)

A42_SA74_Regge.c with A42_SA74_Regge_Driver.py backs the Regge paragraph of Section 8.7 (`gcc -O2 -o regge A42_SA74_Regge.c -lm`, then

`python3 A42_SA74_Regge_Driver.py`; about 30 minutes on two cores; results in A42_SA74_Regge_Results.json). It samples Euclidean Regge gravity, I = lambda *sum V - k* sum (deficit angle *area), lambda = 1, uniformly in the squared edge lengths on periodic Freudenthal triangulations of 3^4 and 4^4 sites, by single-edge Metropolis updates plus exact global rescaling moves. Self-tests: cell counts per vertex (15, 50, 60, 24), deficit angles of the flat start (below 2e-15), the regular 4-simplex's dihedral angle arccos(1/4), angles and volumes against explicit embeddings, the Schlaefli identity, and incremental against full recomputation. Each run reports the average curvature R = 2 sum (deficit* area) / sum V with blocked errors, mean squared edge lengths, acceptance and the scaling identity <2 lambda sum V - k sum (deficit * area)> = number of edges.